



University College Roosevelt

Lange Noordstraat 1, Middelburg, The Netherlands

Bachelor of Liberal Arts and Sciences

Senior Project

JUCE vs VST3 SDK

Measuring Framework Overhead in Audio Plugins: A Comparative Study

Author:

R. Rancāns

2684578 (Student Number)

Supervisor:

T. Vinkó

Second Examiner:

Prof. F. van der Stappen

May 25, 2026

1 Introduction	2
1.1 History	2
1.2 Problem outline	2
2 Background	4
2.1 Audio Plugin Development Frameworks	4
2.2 Performance Comparisons of Audio Frameworks	4
2.3 Benchmarking Methodology for Audio Software	5
2.4 Hardware Performance Counters in Audio Research	5
3 Methodology and Experimental Setup	6
3.1 Computational Environment	6
3.1.1 Test system specification	6
3.1.2 Audio System	6
3.1.3 Software	6
3.1.4 System State During Measurement	7
3.2 Plugin Implementations	7
3.2.1 DSP Algorithm	7
3.2.2 VST3 SDK Implementation	8
3.2.3 JUCE Implementation	9
3.2.4 Verification	10
3.3 Benchmark Design	12
3.3.1 Experimental Setup	12
3.3.2 CPU Measurements and Benchmarking	13
3.4 Metrics	15
3.5 Statistical Analysis	15
4 Results and Analysis	16
4.1 Cycles	16
4.2 IPC Measurement	18
4.3 Per-plugin CPU Measurement via Reaper's Performance Meter	21
4.4 Comparing the Measurement Sources	22
5 Discussion	24
5.1 Threats to Validity	24
5.2 Implications	25
6 Conclusion and Future Work	26
References	27
Appendix A: Definitions	29
Appendix B: Code	31
B.1: Benchmark code for data collection.	31
B.2: Verification program developed to verify both plugins having the same output for an identical input.	31
B.3: VST3 SDK processor.cpp file section that implements the DSP algorithm.	32
B.4: JUCE PluginProcessor.cpp file, which includes the processBlock() method.	33

1 Introduction

Audio software development for music production today is largely dependent on the plugin infrastructure setup. Software that gets developed is usually within two fields: Digital Audio workstations (DAW) or Audio Plugins, that get used in said DAWs. This paper aims to investigate the problem of the choice of framework for plugin development, given that this field lacks empirical data for evaluation, leading to possible bottlenecks in performance of the developed plugins.

1.1 History

As Wilmering et al. (2020) has described, there was a turning point when most audio studios became digital software based, instead of analog hardware. That point is where audio plugin development really began to blossom. In the early years discussions revolved around sample rate choices for computer languages, block size reduction as well as early discussions of performance evaluation in terms of computation speed, as well as discussions of the first Digital Audio Workstations (Bernardini, 1998). Digital signal processing (DSP) methods were beginning to be established (Casajús-Quirós, 1998). Later the discussion turned into a larger look at the free software movement of the early 2000s (Bernardini, 2004). Which brings us to the current audio effects/software space.

In this early space plugin architectures relied on dedicated hardware DSP cards, with effects implementation targeting specific processor families such as the Motorola DSP56k (Smekal, et al., 2001). Increases in general-purpose CPU performance have since shifted plugin processing to host-based execution, where framework abstractions rather than processor architecture become the dominant factor in implementation overhead. This led to a few standards of plugin architecture being developed (VST, AU, AAX) for different OS platforms. Steinberg developed the VST3 plugin format, which is most widely spread across Windows and somewhat on Linux, making it the industry standard.

1.2 Problem outline

Currently, audio software development of plugins is rooted in already established frameworks of development, most often in C++, however, there are people who develop their own audio software architecture (Mitchell, 2008). Hollemans (2024) points out the two most prominent frameworks for C++ audio plugin development are the JUCE framework and iPlug2

framework. In general, the field of audio plugin framework performance benchmarking is scarce, with only a few notes on the important metrics for plugin performance. One of the metrics of audio effects or audio software in general being latency (Wang, et al., 2012). This is mostly related to the fact that audio software very often is made for musicians that are using it to record live performances or use it in a real-time setting (Bencina, 2011). However, of great importance is also the CPU usage and memory usage (Hiljanen, Karlsson, 2020). CPU usage is of particular interest since audio plugins can often have a heavy CPU load, for example, a plugin has a complex reverberation algorithm as part of its DSP path (Smith, 2010). As such, this paper aims to explore the performance metric of CPU usage and utilization for these two frameworks.

This paper seeks to answer the question: does the JUCE framework impose measurable CPU overhead compared to a functionally equivalent VST3 Software Development Kit implementation when hosted in a real-time DAW environment.

With the goal of answering this question, the paper contributes to the audio plugin performance benchmarking field in a few key ways. First, it provides two verified to be functionally equivalent implementations of audio plugins in both the JUCE framework (targeting the VST3 plugin format) and the VST3 SDK. Second, a methodology for hardware based measurement of performance in audio plugins. Thirdly, a quantitative answer to the question with statistical support.

2 Background

This section covers and reviews prior work, which impacts the decisions made for the methodology of this paper.

2.1 Audio Plugin Development Frameworks

The first widely adopted cross-platform C++ audio Input/Output library was RtAudio (Scavone, 2001), which established the pattern of abstracting platform-specific audio APIs behind a unified interface. Later, more complete application frameworks emerged. JUCE, originally released in 2004, extended this approach to target plugin formats (VST, AU, AAX), parameter management and GUI components within a single API (JUCE, 2024). Mitchell (2008) provides an early survey of the C++ synthesis and plugin development space that illustrates how fragmented the tools for plugin development were before frameworks congregated a user base. Today, JUCE and iPlug2 are considered the two most prominent frameworks for cross-platform plugin development (Holleman, 2024), with the raw VST3 SDK serving as the underlying interface that both ultimately target.

2.2 Performance Comparisons of Audio Frameworks

In spite of the significance of choosing a framework, systematic performance evaluation research in this area is sparse. The most directly related work is that of Hiljanen and Karlsson (2020), who conducted an experiment comparing JUCE and the FAUST domain-specific language for audio plugin development, measuring latency, CPU load, and memory usage across three machines. Their study found that FAUST outperformed JUCE on latency and CPU load, yet JUCE used less memory. They used a delay plugin as their test subject and measured CPU load using the macOS Activity Monitor, recording observations of the values it presented.

This study diverges from the approach of Hiljanen and Karlsson in two important ways. First, the comparison is between JUCE and the VST3 SDK, with the aim to capture overhead - extra computations or computer resource usage - that represents the cost of the JUCE abstraction layer specifically, given that JUCE is built on top of the VST3 SDK. Second, the measurement instrumentation for this study includes a hardware performance monitor `perf`, giving hardware-level information rather than process-level monitored readings.

2.3 Benchmarking Methodology for Audio Software

To benchmark audio plugins or audio in general, a deep consideration should be dedicated to the workload said plugin or piece of audio software should process. Hiljanen and Karlsson (2020) used a delay plugin as the workload, reasoning that it is simple, yet representative of real DSP work. A different approach can be seen in GPU audio research, where Skare (2024) uses almost non-existent processing to isolate framework and API overhead from algorithmic cost, which is an approach referred to as no-op benchmarking. This study utilizes the latter approach, using a gain plugin whose DSP reduces to a single multiplication per sample, ensuring that any measured difference between the two implementations is attributable to the framework infrastructure rather than the algorithm.

Wang et al. (2012) discuss the relationship between buffer size, instance count, and measured latency in multichannel audio evaluation platforms, noting that the signal-to-noise ratio of plugin-level measurement depends heavily on how the workload is scaled relative to the host. This gives reason to consider multiple instances for measurement, given that a single instance is a small cost when compared to host process noise, as discussed in Section 3.3.

2.4 Hardware Performance Counters in Audio Research

The Linux `perf` tool provides access to the CPU's Performance Monitoring Unit, which tracks hardware events such as clock cycles, retired instructions and cache misses directly in the CPU rather than via software sampling (Gregg, 2020). This approach avoids the sampling overhead and imprecision of software profilers, making it well suited for measuring real-time audio workloads. Kelly and Oshana (1998) note that cycle-level instrumentation is necessary for meaningful performance engineering of DSP software, since the relevant differences often appear at the level of pipeline utilization, rather than execution time. Instructions per cycle (IPC), derived from the ratio of instructions retired to cycles, is used here as a complementary metric to raw cycle counts, following standard practice in systems performance analysis (Gregg, 2020).

3 Methodology and Experimental Setup

In this section the computational environment is explained, the implementation of the plugins is documented, the methodology described and the statistical analysis approaches explained. The plugin of choice is a Simple Gain Plugin, for the reasons outlined below. To put it simply a gain plugin is a volume control for incoming audio.

3.1 Computational Environment

3.1.1 Test system specification

All measurements were performed on one computer with the following specification:

- **CPU:** AMD Ryzen AI Max 395, 16 cores, up to 5.1 GHz
- **RAM:** 64 GB
- **Storage:** SSD
- **Operating system:** Linux Mint Cinnamon 22.3
- **Kernel:** Linux 6.18.0-37-generic.

3.1.2 Audio System

To benchmark a plugin, a host is required to run audio through said plugin. For this study the host of choice was the DAW Reaper. Audio routing on the test system is handled by PipeWire, with Reaper connecting through PipeWire's JACK compatibility layer. The following audio parameters were used for all measurements:

- **Sample rate:** 48 kHz
- **Buffer size (Pipewire quantum):** 1024 samples
- **Buffer period:** ~22 ms
- **Audio interface:** onboard PCI audio
- **Channel layout:** stereo (2 channels)
- **Sample format:** 32-bit floating point
- **Plugin format:** VST3

3.1.3 Software

The following versions of software were used for the experiments.

- **Reaper:** v7.61
- **JUCE:** 8.0.12

- **VST3 SDK:** 3.8.0
- **Compiler:** gcc 13.3.0
- **CMake:** 3.28.3
- **PipeWire:** 1.0.5

Both plugins were built in “Release” configuration through CMake to ensure that an optimized version of the builds would be used and no unwanted overhead was to be measured.

3.1.4 System State During Measurement

A few system-level configurations were applied to reduce variance in the measurements and to ensure that observed differences reflect plugin behavior, rather than system state. These configurations were active throughout all measurement runs.

CPU frequency scaling was set to “performance” governor for all CPU cores via the `cpupower` utility with the command `sudo cpupower frequency-set -g performance`. This sets the CPU clock frequency to its (not boosted) maximum. This prevents CPU cycle measurement variation, since the frequency is set to be constant. Without a constant clock frequency, the same number of operations could yield different cycle counts across runs, making comparison unreliable.

Background processes were minimized. Any unrelated application, browser or file manager was closed before measurement. The only running applications were the plugin host Reaper and a terminal that ran the tests.

3.2 Plugin Implementations

This section explains the choice of DSP algorithm used in both plugin implementations. It also documents the individual implementation details of both plugin implementations, and lastly verifies the equivalence of both plugins.

3.2.1 DSP Algorithm

The selection of DSP algorithms for both plugins is of paramount importance, for it can be a great contributor to processor usage. A complex DSP algorithm can often require multiple CPUs working in parallel to complete its calculations, like that of an exact reverberation algorithm (Smith, 2010). As such, the chosen algorithm for both implementations is a trivial gain algorithm:

$$y[n] = x[n] \cdot g,$$

where $x[n]$ is the input sample, g is a scalar gain coefficient and $y[n]$ is the output sample. By selecting this simple per-sample algorithm, it is ensured that the DSP work is reduced to a negligible fraction of the computation. With this, any measurable difference in CPU usage between the two plugin implementations can be attributed to primarily the framework structure that encapsulates the DSP algorithm. This approach is similar to that of a “no-op” benchmarking process in other audio performance studies, where empty kernels are used to isolate API and runtime overhead from algorithmic cost (Skare, 2024). Having selected a simple DSP algorithm, it is assured that performance measurements are measuring framework overhead, which simplifies the implementation of each plugin as well.

3.2.2 VST3 SDK Implementation

The VST3 SDK requires the plugin to implement two distinct components that together constitute a plugin instance: an audio processor and an edit controller (Steinberg Media Technologies, 2026). The audio processor is responsible for all real-time signal processing and is implemented by subclassing `Steinberg::Vst::AudioEffect` and overriding the `process` method, which is invoked by the host once per audio buffer. The edit controller handles parameter management and host-facing parameter metadata and is implemented by subclassing `Steinberg::Vst::EditController`. Communication between the two components occurs through the host, i.e. a DAW like Reaper, rather than directly. The plugin also must define and register its component identifiers, audio bus configuration (a stereo input and output), and parameter set (a normalized gain coefficient and a bypass flag). Several constraints apply to the implementation of the `process` method to ensure compatibility with real-time audio processing. For example, memory must not be dynamically allocated within the audio thread (Bencina, 2011), all required buffers and state are allocated during `setupProcessing`, which the host invokes before audio processing begins. Parameter changes arrive as a queue of timestamped values within the `ProcessData` structure passed to each `process` call, and must be applied before the corresponding samples are processed. Silence flags supplied by the host indicate buffers in which the input is known to be zero, in which case the host may pass null buffer pointers. The implementation must handle this case explicitly to avoid dereferencing null memory.

Most of these requirements are satisfied by the Steinberg provided “Hello World VST3” example project (Steinberg Media Technologies, 2025), which was used as a starting point for

the precise implementation of the desired simple gain plugin. Two parameters were added: a gain parameter and a bypass parameter, where the former specifies the applied scalar value of g and the latter defines a toggle switch for disabling the DSP algorithm. As described above, the audio processing happens in the `process` method, which is implemented in a `processor.cpp` file. Parameter changes arriving mid-buffer are handled by reading the last point from the parameter change queue before the sample loop begins. The sample loop is defined as follows:

```
while (--samples >= 0)

    *ptrOut++ = (*ptrIn++) * gain;
```

The pointers are obtained by the `ProcessData` structure.

The plugin has no implementation of GUI to avoid unnecessary interruptions in testing. `CreateView` is used for GUI, but in this implementation `CreateView` returns a null pointer, meaning that no graphical user interface is drawn on screen.

3.2.3 JUCE Implementation

Implementing a plugin using the JUCE framework is considerably simpler, as it is made to simplify the SDK architecture to a usable two-file class structure (JUCE, 2024). The benefit of this abstraction layer is that of less verbose code.

A JUCE plugin is implemented by subclassing `juce::AudioProcessor` and overriding `processBlock`, which the framework invokes once per audio buffer. The `processBlock` method receives a `juce::AudioBuffer<float>` containing both input and output channel data and a `juce::MidiBuffer` for MIDI events (MIDI is defined in Appendix A). For non-MIDI plugins like the desired gain plugin, the MIDI buffer is unused but it must still be present, as required by the base class implementation. Parameters are exposed to the host through `juce::AudioParameterFloat` for gain and `juce::AudioParameterBool` for bypass. These parameters are declared via `addParameter`. The JUCE framework handles parameter normalization, denormalization on access, automation and host-side metadata internally, removing the need to implement a separate edit controller as is required by the VST3 SDK implementation. This introduces a small amount of per-access arithmetic not present in the raw SDK plugin. When building the JUCE plugin, the plugin format is specified as VST3.

For this implementation the same real-time audio constraints apply here, as they did for the VST3 SDK version: no dynamic memory allocation within the `processBlock` and any state required at runtime is allocated in `prepareToPlay`, which JUCE invokes before audio processing begins. The DSP algorithm implementation is functionally the same as the loop in the VST3 SDK implementation, despite the differences in the surrounding constructs:

```
const float g = gain -> get();

for (int channel = 0; channel < buffer.getNumChannels(); ++channel)
{
    auto* data = buffer.getWritePointer(channel);

    for (int i = 0; i < buffer.getNumSamples(); ++i)
        data[i] *=g;
}
```

3.2.4 Verification

This section explains how verification of both plugin algorithms being equal was done, as this verification guarantees that both plugins perform exactly equivalent operations on the audio. With this verification a claim can be made that the measured differences in performance are related to framework overhead, not algorithm cost.

To establish that both plugin implementations produce the same output for the same input, a verification script was executed, comparing the outputs of the two DSP algorithms sample by sample. Without this confirmation, any observed differences in performance between the plugins could not be attributed to framework overhead alone, since it could equally reflect a difference in what is being computed. The important distinction here is that what is being compared is the DSP algorithm, i.e. the per-sample arithmetic, rather than the plugins as complete entities. For clarity, the DSP algorithm is exactly the same in both plugins, as explained in Section 3.2.1, but the exact code used is different and can be seen in Figure 3.1. The surrounding framework code is what will be examined further in this paper.

```

void processSDK(float* in, float* out, int samples, float gain){
    for (int i = 0; i < samples; ++i)
        out[i] = in[i] * gain;
}

void processJUCE(float* data, int samples, float gain){
    for (int i = 0; i < samples; ++i)
        data[i] *= gain;
}

```

Figure 3.1: Both the JUCE and VST3 SDK implementations of the DSP algorithm for verification.

The verification was performed in a standalone C++ program that takes the per-sample DSP algorithm implementation from each plugin and executes both on identical input buffers. The structure of these implementations differs, but functionally should be exactly the same. In the VST3 SDK implementation, the sample loop iterates over channels in the outer loop and samples in the inner loop, operating on raw pointers obtained from the host's `ProcessData` struct. In the JUCE implementation, the equivalent loop uses `buffer.getWritePointer(channel)` to obtain a pointer to each channel's data, then iterates over samples. In both cases, the actual arithmetic is identical – a single multiplication of each sample by the gain value – but the surrounding code differs because each framework has its own conventions for how audio data is presented to the plugin. Specifically, the VST3 SDK exposes channel buffers as an array of `void**` pointers that must be cast to `Sample32*`, while JUCE wraps the same underlying memory in an `AudioBuffer<float>` object that provides typed accessor methods.

The standalone program generated a 440 Hz sine wave at a sample rate of 44.1 kHz as input and processed it through both extracted implementations. Output buffers were compared sample by sample and were found to be bit-exact. For every input sample, the 32-bit floating-point output of the SDK implementation was equal in every bit to that of the JUCE implementation.

The bit-exact result is considered, since the IEEE 754 standard for floating-point arithmetic specifies that a correctly rounded multiplication of two single-precision values produces a uniquely determined result on any conforming implementation (IEEE, 2019). Since the DSP implementation in both plugins consists of one such multiplication per sample, both implementations are required by the standard to produce identical output for the identical

input. The empirical verification confirms that this requirement holds in practice and that no compiler-introduced difference is recorded in further testing of these frameworks.

3.3 Benchmark Design

In this section the methodological design for the benchmarks is outlined and explained. A thorough description of the procedure that was used for benchmarking is given.

3.3.1 Experimental Setup

To measure a statistically significant amount of data, a number of different measurements were made. Given that there are two different implementations and different possible instance counts for each measurement, the experiment's independent variables became: the framework used for the implementation and the number of instances loaded in Reaper.

- Framework used: JUCE or VST3 SDK
- Instance count: 100, 200, 500, 1000 (initially 10, 20, 50, 100)

A pilot run with smaller instance counts {10, 20, 50, 100} was performed as an initial investigation, which showed the necessity for larger instance counts to be used. An observation was made that raised the issue with the chosen benchmarking strategy. The initial data showed almost no difference in the dependent variables. This led to a methodological decision to increase the instance count to better represent what impact the plugin instances have on the Reaper process.

For each combination of the independent variables an output of four different dependent variables is recorded: cpu clock cycles, instructions retired, cache-misses and task-clock. Two of these variables are used to calculate the instructions per cycle (IPC) dependent variable, which is derived from simple division.

$$\frac{\text{instructions}}{\text{cycles}}.$$

The cache-misses and task-clock variables were left out of the analysis, because of time limitations. The measurements were run 20 times for 30 seconds each, to provide a good sample for later statistical analysis described in Section 3.5.

3.3.2 CPU Measurements and Benchmarking

CPU usage was measured using the Linux `perf` tool, which reads hardware performance counters from the CPU's Performance Monitoring Unit (PMU), a dedicated unit in the CPU that tracks events from the CPU, such as clock cycles, instructions retired and cache references (Gregg, 2020). Since the PMU is a part of the CPU itself, using it to measure a process like Reaper should yield satisfyingly accurate results of CPU usage for a real-time audio case. Additionally, another method of confirming CPU usage was recorded using the Reaper Performance Meter.

For each benchmark, `perf` was used to track the Reaper process by its process ID, where the following hardware events were collected: cycles, instructions, cache-misses and task-clock.

Two notable things to mention about CPU cycles. First, it reports all CPU clock ticks recorded for the Reaper process, which means it is not tracking work performed by the plugin in isolation. Second, the cycle counter can be sensitive to CPU frequency scaling (at variable clock frequencies, the same amount of work can produce different cycle counts), which is why the CPU governor was set to "performance" mode (Section 3.1.4).

Reaper's performance meter reports per-effect CPU usage as a percentage of total available CPU. A screenshot of the Performance Meter can be seen in Figure 3.2. It has a value for current CPU usage by the whole Reaper process, as well as individual rows for different tracks in the project (in this case just one track, where white noise is playing), where the columns describe the details of each track. The details of interest are FX CPU, # FX, which are the percentage of CPU usage that all the effects on the track are using (0.24% out of 0.67% in Figure 3.2) and the number of effects/plugin instances on that track.

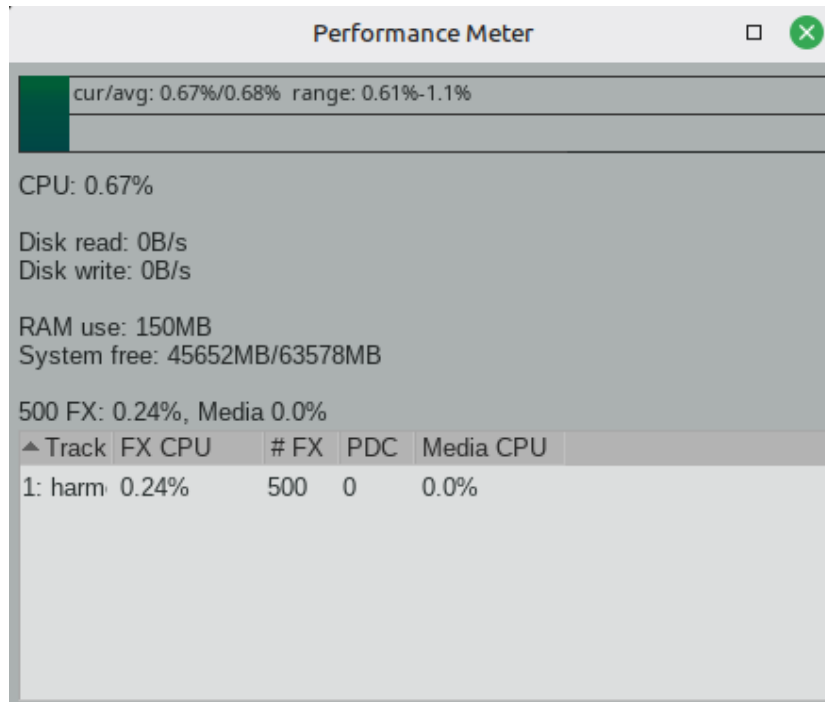


Figure 3.2: Performance Meter running inside of a Reaper project that has 500 instances of the VST3 SDK implementation variant of the plugin running on a track of white noise.

A simple procedure was developed to ensure equal benchmark conditions and simple, error free data collection. The procedure can be outlined in 6 steps:

1. The appropriate Reaper project is opened (with corresponding plugin and instance count).
2. Audio playback was started on a fixed input signal: a white noise .wav file of 30 seconds generated using an online noise generator (Harmonoise, n.d.). The sound is looped continuously.
3. Playback was allowed to stabilize for approximately 30 seconds before running the `benchmark.sh` file. This allows the program to reach a steady state.
4. The measurement script (`benchmark.sh`) is executed and it collects data using `perf` on the Reaper process for 10 minutes (20 runs of 30 seconds).
5. While the test is running, every 2 minutes starting at 0:00 a note is taken of the CPU load (FX CPU) in Reaper's Performance Meter.
6. The `benchmark.sh` script records data from `perf` in a .csv file.

This procedure is executed 8 times for all pairings of the independent variables. The choice to run all 20 runs back-to-back in one single project session was to minimize variance caused by system-state.

3.4 Metrics

Cycles - represents the number of unhalted CPU clock cycles consumed by the tracked process. This is the primary metric of interest, as it represents the total CPU work performed by the host process during the measurement window.

Instructions - instruction events track the number of instructions retired by the CPU during the measurement window. Combined with cycles, this can be used to calculate IPC, which would measure the pipeline's efficiency.

FX CPU - The percentage of total CPU time attributed to the effects (plugins) chain on a track, as reported by Reaper's Performance Meter. With this an isolated measuring of CPU usage by the plugin instances can be captured.

3.5 Statistical Analysis

To determine whether the observed differences in CPU cycle counts and instruction counts between the two plugin implementations are statistically meaningful, rather than the results of random variation, statistical analysis should be performed.

For each instance count condition, an independent-samples Welch's t-test was performed comparing 20 measurement runs of the SDK implementation against the 20 runs of the JUCE implementation. The t-test was chosen because the two implementations are independent of one another, since they were measured in separate Reaper sessions, and because the sample size of 20 runs per condition is sufficient to justify the normality assumptions underlying the test. A significance threshold of $\alpha = 0.05$ was applied to all comparisons.

These tests were applied to both the cycle count data (Section 4.1) and the instruction count data (Section 4.2). The results of these tests are reported alongside the descriptive statistics in Tables 4.1 and 4.2.

4 Results and Analysis

The following results represent the empirically collected data during the study, with appropriate analysis, illustrating the implications of the benchmarks.

4.1 Cycles

To begin the measurement of CPU usage of both plugin implementations an initial run of the benchmark was done with instance counts in the range $\{10, 20, 50, 100\}$. The output of the benchmark can be seen in Figure 4.1, where CPU cycles are illustrated per instance count.

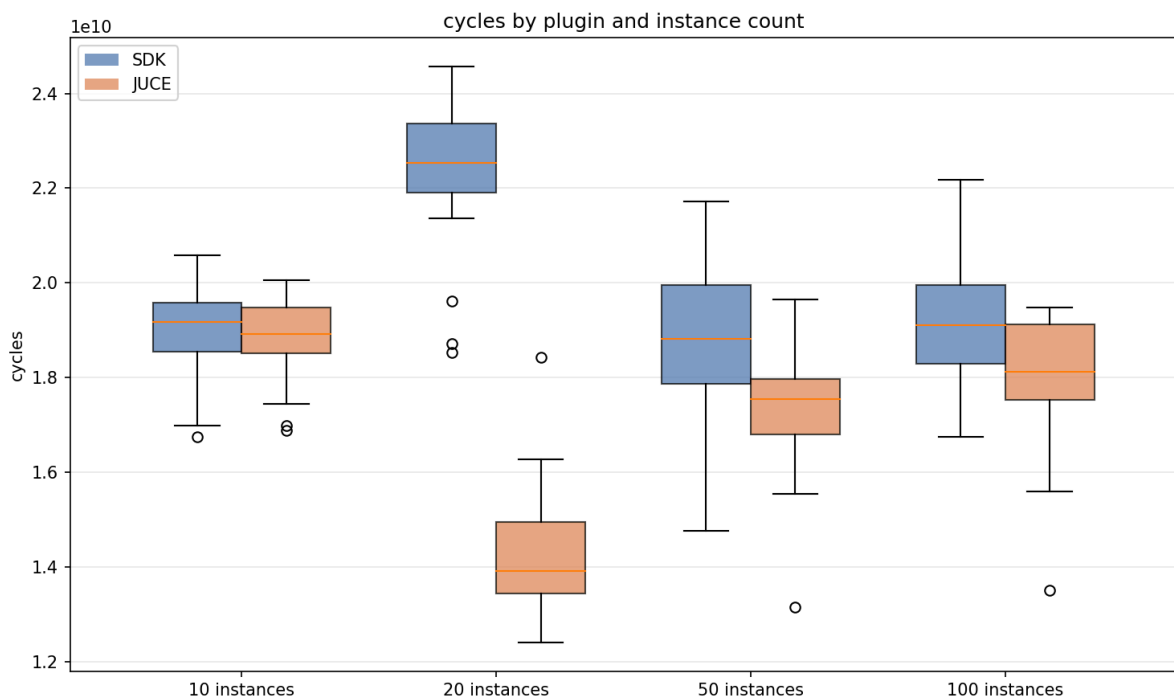


Figure 4.1: CPU cycles used by the JUCE and VST3 SDK plugin implementations per instance count.

These results gave reason to consider larger instance counts, as described in Section 3.3.1, which are considered next.

Twenty measurement runs of 30 seconds each were collected at four instance counts (100, 200, 500, 1000) for both the raw VST3 SDK and JUCE implementations. Figure 4.2 shows the distribution of cycle counts across all conditions. At 100 instances, JUCE consumed substantially more cycles than the SDK, with a visible difference between the two distributions. Additionally it can be observed that the measurements for JUCE at 100 instances seem to have had a huge variance with four significant outliers and a significantly

large inter quartile range. At 200 instances, the two distributions overlap closely. At 500 instances, JUCE again shows higher cycle counts than the SDK. At 1000 instances, the two implementations similarly differ in cycle counts as in the 100 instance and 500 instance count, with JUCE taking seemingly more cycles than the SDK.

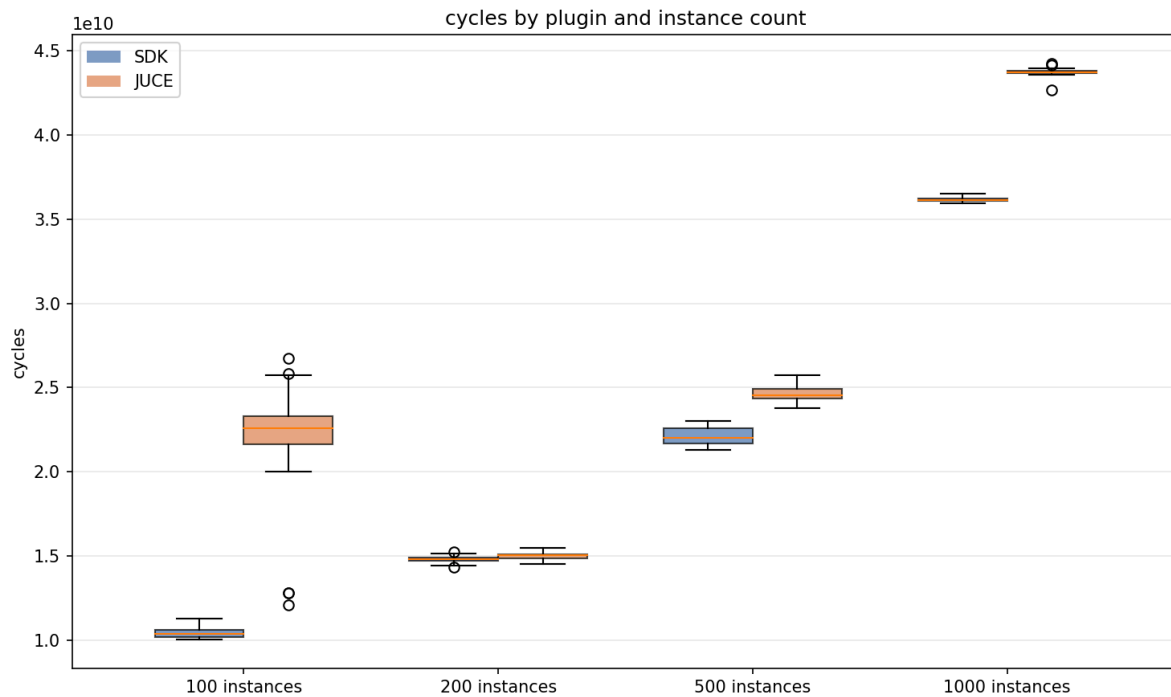


Figure 4.2: CPU cycles used by the JUCE and VST3 SDK plugin implementations for higher instance counts.

A relatively clear pattern emerges from Figure 4.2, where the JUCE implementation consistently performs an equal amount or more CPU cycles, than the VST3 SDK. Yet there is a large discrepancy in the data at 100 instances that must be examined. It implies that the JUCE framework adds an inconsistent amount of CPU cost, however the expectation was for the framework to impose a fixed per-instance overhead amount. The assumption then would be leading to an approximately linear relationship between instance count and measured CPU cycle count. A possible explanation is based on what perf cycles actually measure in this context that being the whole Reaper process, which is discussed in Sections 3.3.2, 4.4 and 5.1.

The statistical comparison of cycle counts between two implementations is summarized in Table 4.1. At 100 instances, the JUCE implementation consumed a mean of 2.15×10^{10} cycles compared to 1.05×10^{10} for the SDK meaning a 106% difference. At 200 instances the difference shrinks to 1.3% considerably though a t-test still gives a statistically significant result with a p value of 0.011. At 500 instances the gap is larger again at 11.4%, while at 1000 instances JUCE uses approximately 20.9% more CPU cycles than the SDK. Every pairwise comparison is statistically significant at a $\alpha = 0.05$ level.

Instances	SDK mean cycles	JUCE mean cycles	% difference	p
100	1.05×10^{10}	2.15×10^{10}	+106%	<0.001
200	1.48×10^{10}	1.50×10^{10}	+1.3%	0.011
500	2.21×10^{10}	2.46×10^{10}	+11.4%	<0.001
1000	3.61×10^{10}	4.37×10^{10}	+20.9%	<0.001

Table 4.1: T-test results for CPU cycle counts between SDK and JUCE implementations.

Because both cycle and instruction counts changed between frameworks, the ratio of the two (IPC) was examined to understand whether the difference reflects more work being done or the same work being done less efficiently.

4.2 IPC Measurement

IPC is a commonly used processor efficiency metric that shows how many instructions the CPU retires on average during each clock cycle. Higher IPC values generally imply more efficient utilization of the processor pipeline, while lower values can indicate inefficiencies (Gregg, 2020). Since the DSP algorithm is the same for both frameworks, any differences in IPC would suggest differences in how efficiently each framework's surrounding infrastructure interacts with the CPU execution pipeline.

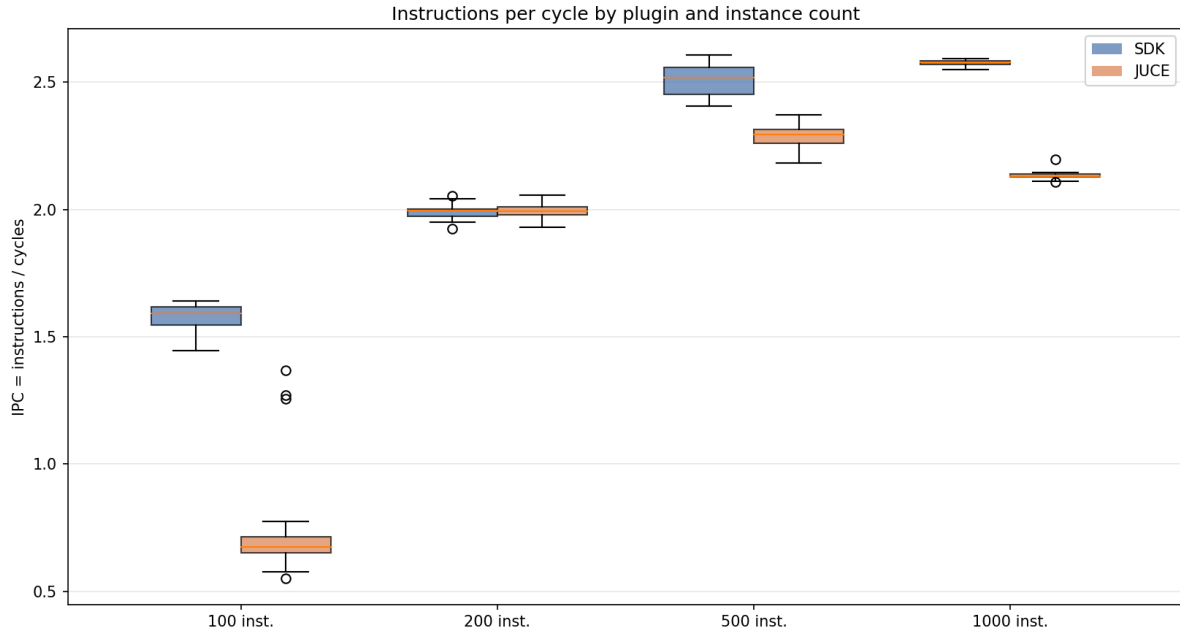


Figure 4.3: IPC measurements observed for large instance counts by plugin type (JUCE, VST3 SDK).

Figure 4.3 illustrates the IPC distributions for both implementations across all tested instance counts. At 100 instances, the VST3 SDK implementation shows substantially higher IPC values than the JUCE implementation. The SDK measurements cluster at around 1.6 IPC, whereas the JUCE implementation centers around 0.7 IPC, with several visible outliers. This suggests that at low instance counts the JUCE implementation retires fewer instructions per CPU cycle and therefore utilizes the processor pipeline less efficiently under these measurement conditions.

At 200 instances, the IPC values for both implementations converge almost completely, with both centered near 2.0 IPC and showing similar variance. This indicates that at this workload the processor executes both implementations with nearly identical efficiency, assuming that the Reaper process (excluding the plugin instance processing) is retiring a constant amount of instructions.

At 500 instances, the SDK implementation again exhibits consistently higher IPC values than the JUCE implementation. Interestingly, at 1000 instances the largest IPC gap can be observed, where JUCE IPC actually drops in value, while the SDK keeps climbing.

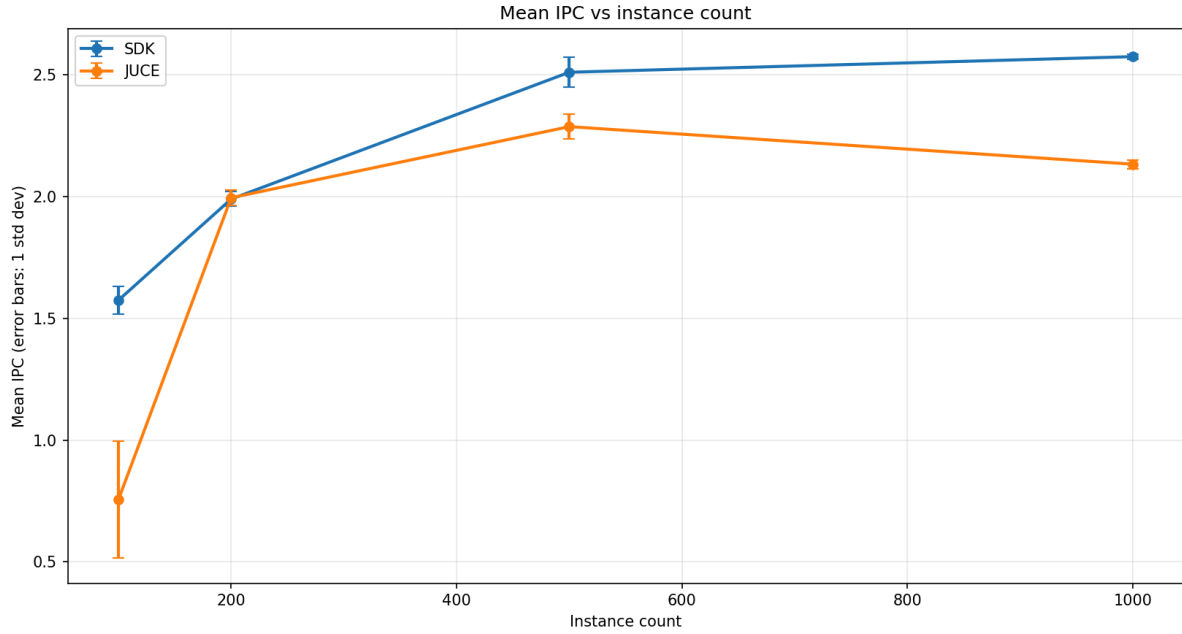


Figure 4.4: Mean IPC values per instance count.

The trend is further illustrated in Figure 4.4, which plots the mean IPC values with one standard deviation error bars. The SDK implementation shows a relatively monotonic increase in IPC as instance count increases, suggesting that the processor pipeline becomes more efficiently utilized as the workload scales. The JUCE implementation also improves substantially from 100 to 500 instances but then decreases slightly at 1000 instances. These results point to the JUCE framework becoming less efficient at utilizing system resources at higher loads on the CPU.

This is further supported by the statistical analysis of the instructions data. Table 4.2 shows that the total instruction counts for both implementations are remarkably close across all conditions, with the biggest difference at 6.8% happening at 100 instances. Since the number of instructions retired is nearly identical between both implementations of the plugins, the IPC difference observed at low instance counts must arise from the SDK implementation retiring those instructions in fewer cycles, not from doing less work.

Instances	SDK instructions	JUCE instructions	% difference	p
100	1.65×10^{10}	1.53×10^{10}	-6.8%	<0.001
200	2.95×10^{10}	2.99×10^{10}	+1.6%	<0.001
500	5.55×10^{10}	5.63×10^{10}	+1.5%	<0.001
1000	9.31×10^{10}	9.33×10^{10}	+0.2%	<0.001

Table 4.2: T-test results for the instructions retired per instance count and plugin implementation (JUCE, VST3 SDK).

Nevertheless, the measurements consistently indicate that the VST3 SDK implementation achieves equal or higher IPC values than the JUCE implementation across most tested conditions.

4.3 Per-plugin CPU Measurement via Reaper’s Performance Meter

To complement the findings from the sections above, an additional source of CPU usage metrics was recorded directly from Reaper’s built-in Performance Meter.

During each measurement run, as explained in Section 3.3.2, a note of the FX CPU metric was taken. The observations are summarized in Table 4.3.

Across all instance counts the JUCE implementation shows higher CPU usage than the VST3 SDK implementation. At 100 instances, the JUCE implementation consumed approximately 1.73 times as much CPU than the SDK implementation according to the Performance Meter. At 200 instances the difference is considerably smaller with both plugins utilizing about the same amount of CPU. At 500 and 1000 instances, the JUCE implementation is again using a considerably larger amount of CPU at 23% and 32% more respectively.

Another observation can be made, when comparing this method with the `perf` cycle measurements, which is the fact of this data being monotonic, in other words, increasing consistently with more plugin instances being measured. This is a welcome change, as it aligns with the expected computational consequences of doing more audio processing.

Instances	Individual measurements of the FX CPU metric for VST3 SDK	VST3 SDK mean	Individual measurements of the FX CPU metric for JUCE	JUCE mean	JUCE/VST3 SDK Ratio
100	0.06, 0.05, 0.04, 0.04, 0.04, 0.04	0.045	0.04, 0.09, 0.11, 0.09, 0.09, 0.05	0.078	1.73
200	0.07, 0.08, 0.08, 0.08, 0.10, 0.09	0.083	0.10, 0.09, 0.08, 0.10, 0.09, 0.09	0.092	1.11
500	0.18, 0.18, 0.22, 0.18, 0.21, 0.20	0.195	0.24, 0.29, 0.23, 0.22, 0.23, 0.23	0.240	1.23
1000	0.53, 0.53, 0.52, 0.53, 0.53, 0.52	0.527	0.71, 0.70, 0.70, 0.68, 0.69, 0.70	0.697	1.32

Table 4.3: Reaper Performance meter observations of total FX CPU usage for SDK and JUCE plugin implementations.

It is important to note however, that the values measured here are extremely small. Even at the highest instance count of 1000 CPU usage remained below 1% of the available CPU resources.

4.4 Comparing the Measurement Sources

The two measurement sources seem to indicate the same finding that the JUCE implementation of the plugin is significantly more CPU usage heavy. The difference lies in rigor and accuracy.

The `perf` cycles data, in spite of being rigorously collected with many repeats at hardware level, suffers from low sensitivity to instance differences because most of the cycles it measures are of the Reaper host process. Having a tiny contribution to the total measurement, the plugin overhead measurement is significantly hindered.

In contrast, Reaper's performance meter isolates each plugin's contribution to CPU usage from the rest of host overhead by design. Though the rigor used in collection of data from the performance meter lacks in comparison to the `perf` data, as it was manual, rather than scripted. Though it illustrates a much clearer picture from the get-go. The difference between JUCE and SDK is shown consistently across instance counts.

These two sources yield a thorough and self-supporting view of how the JUCE framework contributes measurable, yet small overhead to the CPU, when compared to the raw VST3 SDK directly.

5 Discussion

Given the limited amount of research on this topic, the methods used in this study should be carefully scrutinized to achieve a great balance of usability of the methods as well as clear understanding of the shortcomings of said methods. This section explores the threats to validity that the study may suffer as well as the implications of the findings.

5.1 Threats to Validity

There are a few points to consider when considering the validity of the research presented in this paper. An important one, as mentioned in Section 4, the measurements made, in the context of the benchmarks performed, yielded extremely small CPU values. This, of course, is a consequence of choosing a simple DSP algorithm, yet it does limit the analysis to only plugins that don't use complex DSP. Moreover, it leaves the possibility open for more complex DSP plugins to shrink the framework overhead ratio. With the values being this small the usage of `perf` for measurement of CPU cycles, instructions retired and IPC is likely compromised to a great extent. This shortcoming of the study may be remedied by the suggested future work in Section 6.

Another threat to validity would be the usage of only one computer for all the benchmarking, which can be worsened by the possibility of different processes creating noise in the data. Having multiple test machines would make the paper more representative of the different computers that might run such plugins. Although an attempt to mitigate host noise was made, as described in Section 3.1.4.

A different threat related to the computational environment is that of only one DAW (Reaper) being tested. It is a possibility that different DAWs implement host side optimizations for plugins that would bias the results observed in the paper.

Possibly less impactful, yet still worthy of mention, is the fact that all of the plugins were run under a gain setting of 1.0, which leaves the audio at the same volume as it passes through the plugin. If a test of, say, gain value of 0.5 would have been included, perhaps an important observation could have been made regarding the DSP algorithm impact on framework overhead. But no such measurements were made in this study.

Lastly, the fact that 1000 instances were measured is a complete misrepresentation of actual use cases of plugins. Most modern workflows use a few instances per track of audio, leading to misrepresentation of the actual impact of the measured framework overhead.

5.2 Implications

With the limitations and threats to validity in mind, a developer interested in developing plugins should not base their choice of framework from the results for CPU performance alone. The difference for a developer here presents no worry at expected instance counts (~1-10), since the absolute difference in framework overhead is negligible. Overhead exists for simple plugins, but choosing between the ease of use of JUCE or the full control of the VST3 SDK, should be decided by the developer's preference.

6 Conclusion and Future Work

It can be concluded that the JUCE framework does impose measurable and significant CPU overhead when compared to the verified equivalent implementation in the VST3 SDK when hosted in a real-time DAW environment. Furthermore, IPC and instruction analysis, especially at 1000 instances, shows clear hardware-level evidence of the JUCE pipeline inefficiency. These claims are supported by the statistical analysis done in Section 4. The measurable overhead is largely present at high instance counts of a simple DSP plugin implementation. The design of the methodology for this study proved proficient in illuminating the underlying software bottlenecks that could be incurred by framework overhead. Yet there are many considerations to take from this paper, for possible future additional research to broaden the applicable target population.

A suggestion for future work is to perform more benchmarks involving different levels of DSP complexity on multiple different machines and software environments. For other platforms than Linux, a consideration of the iPlug2 framework could also be included. An important limitation in this project was the lack of a standalone plugin host that could be used for benchmarking, so it is a suggestion to consider building such a host for future iterations of research like this.

References

- Bencina, R. (2011). *Real-time audio programming 101: time waits for nothing*. [Blog post]. <http://www.rossbencina.com/code/real-time-audio-programming-101-time-waits-for-nothing>
- Bernardini, N., & Rocchesso, D. (1998). Making Sounds with Numbers: A Tutorial on Music Software Dedicated to Digital Audio. *Proceedings of DAFx-1998 - Barcelona*.
- Bernardini, N., Cirotteau, D., Ekanayaka, F., & Glorioso, A. (2004). Making Sound With Numbers, Six Years Later. *Proceedings of DAFx-2004 - Naples*.
- Casajús-Quirós, F. J. (1998). Digital Signal Processors for real-time audio processing. *Proceedings of DAFx-1998 - Barcelona*.
- Gregg, B. (2020). *Linux perf examples*. <https://www.brendangregg.com/perf.html>
- Harmonoise. (n.d.). *Noise generator* [Web tool]. <https://harmonoise.com/tools/noise-generator>
- Hiljanen, H., & Karlsson, J. (2020). JUCE vs. FAUST : En jämförande studie i prestanda mellan plugins (Dissertation). Retrieved from <https://urn.kb.se/resolve?urn=urn:nbn:se:hj:diva-50355>
- Hollemans, M. (2024, June 22). *Getting Started with Audio Programming: Books and other learning resources for new audio developers*. [audiodev.blog](https://audiodev.blog/newbie-resources/). <https://audiodev.blog/newbie-resources/>
- IEEE. (2019). *IEEE Standard for floating-point arithmetic* (IEEE Std 754-2019). Institute of Electrical and Electronics Engineers. <https://doi.org/10.1109/IEEESTD.2019.8766229>
- JUCE. (2024). *JUCE: The C++ framework for audio applications and plugins*. Raw Material Software Limited. <https://juce.com/>
- Kelly, D. P., & Oshana, R. S. (1998). Software performance engineering a digital signal processing application. *Association for Computing Machinery*. <https://doi.org/10.1145/287318.287328>
- Mitchell, D. R. (2008). *BasicSynth: Creating a Music Synthesizer in Software*. Lulu.com

- Scavone, G. P. (2001). RtAudio: A Cross-Platform C++ Class for Realtime Audio Input/Output. *Center for Computer Research in Music and Acoustics. Proceedings of ICMC 2001*.
- Skare, T. (2024). General-Purpose GPU Audio Benchmark Framework. *Proceedings of DAFx-2024 - Guildford*.
- Smekal, Z., Schimmel, J., & Krkavec, P. (2001). Optimizing Digital Musical Effect Implementation For Harvard Dsp Architecture. *Proceedings of DAFx-2001 - Limerick*
- Smith, J. O., III. (2010). Artificial reverberation. In *Physical audio signal processing for virtual musical instruments and audio effects*. W3K Publishing.
https://ccrma.stanford.edu/~jos/pasp/Complexity_Exact_Reverberation.html
- Steinberg Media Technologies. (2026). *VST 3 SDK* (Version 3.8.0) [Source code]. GitHub.
<https://github.com/steinbergmedia/vst3sdk>
- Steinberg Media Technologies. (2025). *vst3_example_plugin_hello_world: Hello World VST 3 example plug-in* [Source code]. GitHub.
https://github.com/steinbergmedia/vst3_example_plugin_hello_world
- Wang, Y., Zhu, X., & Fu, Q. (2012). A low latency multichannel audio processing evaluation platform. *Audio Engineering Society Convention Paper 8678*.
https://www.researchgate.net/publication/268350397_A_low_latency_multichannel_audio_processing_evaluation_platform
- Wilmering, T., Moffat, D., Milo, A., & Sandler, M. B. (2020). A History of Audio Effects. *Applied Sciences*, 10(3), 791. <https://doi.org/10.3390/app10030791>

Appendix A: Definitions

Audio buffer - A fixed-size array of audio samples passed to a plugin by the host of processing during a process call. The size of a buffer is measured in samples and determines how many samples a plugin can process at once.

Channel - A single stream of audio samples. Stereo audio, for example, has two audio channels: left and right.

CPU clock cycle - The unit of work performed by a CPU.

CPU governor - A Linux kernel component that controls the operating frequency of CPU cores.

DAW (Digital Audio Workstation) - Software that can be used for recording, editing and producing audio or music. For audio plugins, the DAW is the host. The DAW used in this paper is Reaper.

DSP (Digital Signal Processing) - The computational manipulation of digital signals.

FX CPU - The percentage of total CPU time attributed to the effects (plugins) chain on a track, as reported by Reaper's Performance Meter.

Host - The application that loads and runs audio through a plugin. In this paper it is Reaper.

JUCE - An open-source, cross-platform C++ framework for audio application and plugin development. It abstracts the differences between plugin formats (VST3, AU, AAX, LV2) and operating systems behind a unified API.

MIDI (Musical Instrument Digital Interface) - A communication protocol for transmitting musical event data such as notes and parameter changes between devices and software.

perf - A Linux command-line tool that provides access to the CPU's Performance Monitoring Unit (PMU).

Plugin - A software component that runs inside a host application (DAW) to process or generate audio.

Sample - A single floating-point value representing the amplitude of an audio signal at one point in time.

Sample rate - The number of audio samples captured or processed per second, measured in Hz or kHz.

VST3 (Virtual Studio Technology 3) - An audio plugin standard developed by Steinberg Media Technologies. VST3 defines the interface between a DAW host and an audio plugin through a set of C++ abstract interfaces.

VST3 SDK - The official C++ software development kit provided by Steinberg for implementing VST3 plugins. It provides base classes, interface definitions and build infrastructure required to create a compliant VST3 plugin without a higher-level framework.

Appendix B: Code

B.1: Benchmark code for data collection.

B.2: Verification program developed to verify both plugins having the same output for an identical input.

```
1 #!/bin/bash
2 PLUGIN=$1 # "sdk" or "juce"
3 OUTPUT="results_${PLUGIN}.csv"
4 echo "run,cycles,instructions,cache_misses" > $OUTPUT
5
6 for i in $(seq 1 20); do
7     # perf stat outputs to stderr, capture it
8     RESULT=$(sudo perf stat -p $(pgrep reaper | tail -1) \
9         -e cycles,instructions,cache-misses \
10         sleep 30 2>&1)
11     CYCLES=$(echo "$RESULT" | grep "cycles" | awk '{print $1}' | tr -cd '0-9')
12     INSTR=$(echo "$RESULT" | grep "instructions" | awk '{print $1}' | tr -cd '0-9')
13     CACHE=$(echo "$RESULT" | grep "cache-misses" | awk '{print $1}' | tr -cd '0-9')
14
15     echo "$i,$CYCLES,$INSTR,$CACHE" >> $OUTPUT
16     echo "Run $i done"
17 done
```

```
48 #include <stdio>
49 #include <cstring>
50
51 void processSDK(float* in, float* out, int samples, float gain){
52     for (int i = 0; i < samples; ++i)
53         out[i] = in[i] * gain;
54 }
55
56 void processJUCE(float* data, int samples, float gain){
57     for (int i = 0; i < samples; ++i)
58         data[i] *= gain;
59 }
60
61 int main(){
62     const int BUFFER_SIZE = 512;
63     const float GAIN = 0.5f;
64     const float SAMPLE_RATE = 44100.0f;
65
66     // A test buffer of a sine wave at 440hz.
67     float input[BUFFER_SIZE];
68     for (int i = 0; i < BUFFER_SIZE; ++i)
69         input[i] = sinf(2.0f * 3.14159265f * 440.0f * i / SAMPLE_RATE);
70
71     float sdk_output[BUFFER_SIZE];
72     processSDK(input, sdk_output, BUFFER_SIZE, GAIN);
73
74     float juce_output[BUFFER_SIZE];
75     memcpy(juce_output, input, sizeof(input));
76     processJUCE(juce_output, BUFFER_SIZE, GAIN);
77
78     int mismatches = 0;
79     for (int i = 0; i < BUFFER_SIZE; ++i)
80     {
81         if (sdk_output[i] != juce_output[i])
82         {
83             printf("MISMATCH: at sample %d: SDK=%.10f JUCE=%.10f diff=%.2e\n",
84                 i, sdk_output[i], juce_output[i], fabsf(sdk_output[i] - juce_output[i]));
85             mismatches++;
86         }
87     }
88
89     if (mismatches == 0)
90         printf("PASS: all %d samples are bit-exact.\n", BUFFER_SIZE);
91     else
92         printf("FAIL: %d/%d samples differ.\n", mismatches, BUFFER_SIZE);
93
94     return mismatches == 0 ? 0 : 1;
95 }
```


B.3: VST3 SDK processor.cpp file section that implements the DSP algorithm.

```
62
61 //--- Process Audio-----
60 //--- -----
59 if (data.numInputs == 0 || data.numOutputs == 0)
58 {
57     // nothing to do
56     return kResultOk;
55 }
54
53 int32 numChannels = data.inputs[0].numChannels;
52
51 uint32 sampleFramesSize = getSampleFramesSizeInBytes(processSetup, data.numSamples);
50 void** in = getChannelBuffersPointer (processSetup, data.inputs[0]);
49 void** out = getChannelBuffersPointer (processSetup, data.outputs[0]);
48
47 data.outputs[0].silenceFlags = 0;
46
45 if (data.inputs[0].silenceFlags != 0)
44 {
43     // mark output silence too
42     data.outputs[0].silenceFlags = data.inputs[0].silenceFlags;
41
40     // the plug-in has to be sure that if it sets the flags silence that the output buffer are clear
39     for (int32 i = 0; i < numChannels; i++)
38     {
37         // do not need to be cleared if the buffers are the same (in this case input buffer are
36         // already cleared by the host)
35         if (in[i] != out[i])
34         {
33             memset (out[i], 0, sampleFramesSize);
32         }
31     }
30     // nothing to do at this point
29     return kResultOk;
28 }
27
26 float gain = mGain;
25
24 if (mBypass)
23 {
22     for (int32 i = 0; i < numChannels; i++)
21     {
20         int32 samples = data.numSamples;
19         Vst::Sample32* ptrIn = (Vst::Sample32*)in[i];
18         Vst::Sample32* ptrOut = (Vst::Sample32*)out[i];
17         while (--samples >= 0)
16             *ptrOut++ = *ptrIn++;
15     }
14 }
13 else
12 {
11     for (int32 i = 0; i < numChannels; i++)
10     {
9         int32 samples = data.numSamples;
8         Vst::Sample32* ptrIn = (Vst::Sample32*)in[i];
7         Vst::Sample32* ptrOut = (Vst::Sample32*)out[i];
6         while (--samples >= 0)
5             *ptrOut++ = (*ptrIn++) * gain;
4     }
3 }
2 return kResultOk;
1 }
```

B.4: JUCE PluginProcessor.cpp file, which includes the processBlock() method.

```
41 #include "PluginProcessor.h"
40
39 SimpleGainPlugin::SimpleGainPlugin()
38 : AudioProcessor(BusesProperties()
37   .withInput ("Input", juce::AudioChannelSet::stereo(), true)
36   .withOutput("Output", juce::AudioChannelSet::stereo(), true))
35 {
34   addParameter(gain = new juce::AudioParameterFloat("gain", "Gain", 0.0f, 1.0f, 1.0f));
33   addParameter(bypass = new juce::AudioParameterBool("bypass", "Bypass", false));
32 }
31
30 void SimpleGainPlugin::prepareToPlay(double, int)
29 {
28 }
27
26 void SimpleGainPlugin::releaseResources()
25 {
24 }
23
22 void SimpleGainPlugin::processBlock(juce::AudioBuffer<float>& buffer,
21   juce::MidiBuffer&)
20 {
19   juce::ScopedNoDenormals noDenormals;
18
17   if (bypass->get())
16     return; // Input is already in the output buffer, so there is nothing to do.
15
14   const float g = gain->get();
13
12   for (int channel = 0; channel < buffer.getNumChannels(); ++channel)
11   {
10     auto* data = buffer.getWritePointer(channel);
9     for (int i = 0; i < buffer.getNumSamples(); ++i)
8       data[i] *= g;
7   }
6 }
5
4 juce::AudioProcessor* JUCE_CALLTYPE createPluginFilter()
3 {
2   return new SimpleGainPlugin();
1 }
```